P3031R0
Resolve CWG2561, conversion function for lambda with explicit object parameter

Published Proposal, 2023-11-12

Author:
Arthur O'Dwyer

Audience:
EWG, CWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:
7

Abstract

Let `T` be the closure type of `[](this auto, int){}`. CWG2561 points out that the meaning of `+T()` is unclear. Microsoft (non-conformingly) treats that expression the same as `+[](int){}`. Clang (conformingly) treats the lambda as a generic lambda and `+T()` as ill-formed, but (non-conformingly and awkwardly) accepts `void (*pf)(T, int) = T()`. The (February 2022) P/R of CWG2561 proposes Clang's direction, but this paper proposes either the more user-friendly and consistent direction taken by Microsoft, or else to eliminate the function-pointer conversion for explicit-object lambdas altogether.

Table of Contents

1 Note

2 Background

2.1 Is `[](this auto){}` generic?

2.1.1 Is `this auto` different from `this T`?

2.1.2 Should we allow `(+c)()` to act differently from `c()`?

2.1.3 This lambda can only be passed its own type

3 Implementation experience

4 Options for proposed wording

4.1 Punt for now

4.2 Clang's approach

4.3 MSVC's approach

5 Straw polls taken in EWG, 2023-11-07

1. Note

R0 is the initial, and only, revision of this paper. D3031R0 (draft revision 6 dated 2023-11-03) was discussed by EWG in Kona 2023 (minutes), and EWG wisely decided to take the "punt" option as presented. MSVC's behavior is cool and nice, but requires a design paper if we actually want to pursue it. The post-Kona mailing contains P3031R0 (draft revision 7, i.e., including the note you are now reading) only for the official record; this paper demands no further action from anyone.

2. Background

"Deducing this" allows lambdas to have explicit object parameters. Microsoft and Clang both support the feature already, but with differing semantics.

```
struct Any { Any(auto) { puts("called"); } };

auto a1 = [](int x) { return x+1; };
auto a2 = [](this Any self, int x) { return x+1; };
```

Here `a1`'s closure type is certainly equivalent to

```
template<class T> using Just = T;

struct A1 {
  int operator()(int x) const { return x+1; }
  operator Just<int(*)(int)>() const { return +[](int x) { return A1()(x); }; }
};
```

But it's unclear which of the following two options should correspond to `a2`'s closure type:

```
struct A2_MSVC {
  int operator()(this Any self, int x) { return x+1; }
  operator Just<int(*)(int)>() const { return +[](int x) { return A2_MSVC()(x); }; }
};

struct A2_Clang {
  int operator()(this Any self, int x) { return x+1; }
  operator Just<int(*)(Any, int)>() const { return &A2_Clang::operator(); }
};
```

MSVC's version is friendlier: it allows us to refactor an implicit-object-parameter lambda into an explicit-object-parameter lambda (or vice versa) without changing the type of the expression `+a2`. MSVC also allows us to write a recursive captureless lambda taking itself as a function pointer; this is impossible on Clang, because the function pointer's type would recursively involve its own type.

Before	After

`void f(auto c);`
`auto a1 = [](int x) { return x+1; };`
`f(a1);  // OK`
`f(+a1); // OK`
	`void f(auto pf);`
	`auto a2 = [](this Any, int x) { return x+1; };`
	`f(a2);  // OK`
	`f(+a2); // Error on Clang, OK on MSVC`

`// OK`
`auto fib = [](this auto fib, int x) -> int {`
`  return x < 2 ? x : fib(x-1) + fib(x-2);`
`};`
`int i = fib(5);`
	`// Error on Clang, OK on MSVC`
	`auto fib = [](this int (*fib)(int), int x) {`
	`  return x < 2 ? x : fib(x-1) + fib(x-2);`
	`};`
	`int i = fib(5);`

But if you try `A2_MSVC` as written, you'll find that it is ambiguous:

```
A2_MSVC a2;
a2(1); // ill-formed, ambiguous
```

There are two candidates for this call: (1) call `Any(A2_MSVC)` and then `A2_MSVC::operator()(this Any, int)`, or (2) call `A2_MSVC::operator Just<int(*)(int)>() const` and then the built-in `operator()` on that function pointer. We need new core wording to prefer (1) over (2).

Why don't ordinary (implicit-object-parameter) lambdas suffer from this ambiguity? I think it's because the identity conversion binding `this` always wins over the user-defined conversion to function pointer. But for the explicit-object-parameter lambda `a2`, binding `this` to `this Any self` is *also* a user-defined conversion.

§ 2.1. Is `[](this auto){}` generic?

According to [expr.prim.lambda.general]/6, a lambda becomes "generic" when any of its parameters involve placeholder types — even when the only placeholder type is the explicit object parameter's! In other words, `b` below is technically considered a generic lambda.

```
auto b = [](this auto self, int x) { return x+1; };
```

It is unclear which of the following three options should correspond to `b`'s closure type.

```
struct B_Clang {
  template<class T> int operator()(this T self, int x) { return x+1; }
  template<class T> operator Just<int(*)(T, int)>() const { return &B_Clang::operator(); }
};

struct B_Generic {
  template<class T> int operator()(this T self, int x) { return x+1; }
  template<class T> operator Just<int(*)(int)>() const { return +[](int x) { return B_Gener
};

struct B_MSVC {
```

```
  template<class T> int operator()(this T self, int x) { return x+1; }
  operator Just<int(*)(int)>() const { return +[](int x) { return B_MSVC()(x); }; }
};
```

MSVC's version is friendliest:

```
auto b = [](this auto self, int x) { return x+1; };
int (*pb1)(int) = b; // Generic+MSVC: OK; Clang: error
int (*pb2)(decltype(b), int) = b; // Clang: OK; Generic+MSVC: error
auto pb3 = +b; // MSVC: OK; Generic+Clang: error
```

This suggests that a `this auto` parameter shouldn't suffice to make a lambda "generic." But see the next section.

2.1.1. Is `this auto` different from `this T`?

Zhihao Yuan argues that it is counterintuitive for MSVC to treat these two spellings differently:

```
auto a = [](this auto) {}; // "non-generic" on MSVC
auto b = []<class T>(this T) {}; // generic

auto pa = +a; // OK on MSVC
auto pb = +b; // error on MSVC
void (*qa)() = a; // OK on MSVC
void (*qb)() = b; // error on MSVC
```

If `[]<class T>(this T){}` is considered generic, then so should be `[](this auto){}`; we shouldn't carve out an exception for the latter.

2.1.2. Should we allow `(+c)()` to act differently from `c()`?

Gašper Ažman provides this example of a lambda that can't be called directly:

```
auto c = [](this auto self) -> int { return self.value; };
using C = decltype(c);

struct Derived : C { int value = 2; };
```

Here it is a (SFINAE-unfriendly) error to instantiate `C::operator()<C>`, but it is OK to instantiate `C::operator()<Derived>`. Therefore both `c()` and `+c` are errors, but this is OK:

```
static_assert(std::is_convertible_v<C, int(*)()>); // OK on MSVC
static_assert(std::is_convertible_v<C, int(*)(C)>); // OK on Clang

Derived d;
int two = d(); // OK
int (*p)() = d; // error on Clang+MSVC
int (*p)(Derived) = d; // OK on Clang, error on MSVC
```

On Clang, `+d` fails to deduce the template parameter to `C`'s conversion function template (SFINAE-friendly). On MSVC, `+d` unambiguously calls the non-template conversion function inherited from `C`, which hard-errors during instantiation of `C::operator()<C>`.

A more problematic variation is:

```
auto c2 = [](this auto self) { return sizeof(self); };
struct Derived2 : decltype(c2) { int pad; } d2;
assert(d2() == 4);
assert((+d2)() == 1);
```

Here `d2() == 4`, but `+d2` points to a function that returns `1`. This example suggests that the conversion function should not exist for explicit-object lambdas (i.e., the "Punt" wording option below).

Two other problematic cases are `c3` (no inheritance):

```
auto c3 = [](this auto&& self) { return std::is_rvalue_reference_v<decltype(self)>; };
assert(c3() == false);
assert((+c3)() == true);
```

and `d4` (a non-generic lambda):

```
struct Evil { int i; Evil(auto x) : i(sizeof(x)) {} };
auto c4 = [](this Evil self) { return self.i; };
struct Derived4 : decltype(c4) { int pad; } d4;
assert(d4() == 4);
assert((+d4)() == 1);
```

Maybe these are obscure enough problems that we don't care? Or, on the other hand, maybe we should make a solid rule that *whenever `(+lambda)()` is well-formed, it is guaranteed to have the same behavior as `lambda()`*. Explicit-object lambdas cannot provide that guarantee, and therefore we cannot give them conversion functions (i.e., the "Punt" wording option below).

§ 2.1.3. This lambda can only be passed its own type

Consider this case, which (having a *template-parameter-list*) is clearly a generic lambda:

```
auto c = []<class T>(this T self, T x) { std::cout << x; };
```

It is unclear which of the following three options should correspond to `c`'s closure type:

```
struct C_Clang {
  template<class T> void operator()(this T self, T x) const { std::cout << x; }
  template<class T> operator Just<void(*)(T, T)>() const { return &C_Clang::operator(); }
};

struct C_MSVC {
  template<class T> void operator()(this T self, T x) const { std::cout << x; }
  template<class T> operator Just<void(*)(T)>() const
```

```
  { return +[](auto x) { std::cout << x; }; }
};

struct C_Constrained {
  template<class T> void operator()(this T self, T x) const { std::cout << x; }
  template<class T> operator Just<void(*)(T)>() const
    requires std::is_same_v<C_Constrained, T>
  { return +[](auto x) { std::cout << x; }; }
};
```

Here MSVC's version is friendly, but confusing, because MSVC rightly rejects `c(1)` but accepts `(+c)(1)`! So the function pointer that MSVC returns from `+c` is not in fact "invoking the closure type's function call operator on a default-constructed instance of the closure type" — that wouldn't compile! Do we need the conversion function template to be constrained? and if so, should it be constrained as in `C_Constrained`, or otherwise?

§ 3. Implementation experience

As far as I can tell, we have implementation experience of both Clang's approach (in Clang) and MSVC's approach (in MSVC) — although I don't fully understand what MSVC is doing internally to avoid the overload-resolution ambiguity. But MSVC's approach seems to be implementable, since it's been implemented.

§ 4. Options for proposed wording

> NOTE: Throughout, the Standard's chosen examples rarely seem on-point. I'd like to add more relevant examples and eliminate some of the examples already present.

§ 4.1. Punt for now

Modify [expr.prim.lambda.general]/6 as follows:

> 6. A lambda is a *generic lambda* if the *lambda-expression* has any generic parameter type placeholders ([dcl.spec.auto]), or if the lambda has a *template-parameter-list*.
> [*Example 4:*
>
> ```
> ~~int i = [](int i, auto a) { return i; }(3, 4);                    // OK, a generic lambda~~
> ~~int j = []<class T>(T t, int i) { return i; }(3, 4);              // OK, a generic lambda~~
> auto x = [](int i, auto a) { return i; };                           // OK, a generic lambda
> auto y = [](this auto self, int i) { return i; };                   // OK, a generic lambda
> auto z = []<class T>(int i) { return i; };                          // OK, a generic lambda
> ```
>
> — *end example*]

Modify [expr.prim.lambda.closure]/9 as follows:

9. The closure type for a non-generic *lambda-expression* with no *lambda-capture* and no explicit object parameter ([dcl.fct]) whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage having the same parameter and return types as the closure type's function call operator. The conversion is to "pointer to noexcept function" if the function call operator has a non-throwing exception specification. If the function call operator is a static member function, then the value returned by this conversion function is the address of the function call operator. Otherwise, the value returned by this conversion function is the address of a function `F` that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type. `F` is a constexpr function if the function call operator is a constexpr function and an immediate function if the function call operator is an immediate function.

10. For a generic lambda with no *lambda-capture* and no explicit object parameter, the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list, and the pointer to function has the same parameter types, as the function call operator template. The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization.

11. [*Note 4:* If the generic lambda has no *trailing-return-type* or the *trailing-return-type* contains a placeholder type, return type deduction of the corresponding function call operator template specialization has to be done. The corresponding specialization is that instantiation of the function call operator template with the same template arguments as those deduced for the conversion function template. Consider the following:

```
auto glambda = [](auto a) { return a; };
int (*fp)(int) = glambda;
```

The behavior of the conversion function of `glambda` above is like that of the following conversion function:

```
struct Closure {
  template<class T> auto operator()(T t) const { /* ... */ }
  template<class T> static auto lambda_call_operator_invoker(T a) {
    // forwards execution to operator()(a) and therefore has
    // the same return type deduced
    /* ... */
  }
  template<class T> using fptr_t =
     decltype(lambda_call_operator_invoker(declval<T>())) (*)(T);

  template<class T> operator fptr_t<T>() const
    { return &lambda_call_operator_invoker; }
};
```

— *end note*]

[*Example 6:*

```
void f1(int (*)(int))   { }
void f2(char (*)(int))  { }

void g(int (*)(int))    { }     // #1
void g(char (*)(char))  { }     // #2

void h(int (*)(int))    { }     // #3
void h(char (*)(int))   { }     // #4

auto glambda = [](auto a) { return a; };
f1(glambda);                    // OK
f2(glambda);                    // error: ID is not convertible
g(glambda);                     // error: ambiguous
h(glambda);                     // OK, calls #3 since it is convertible from ID
int& (*fpi)(int*) = [](auto* a) -> auto& { return *a; };        // OK
```

— *end example*]

12. If the function call operator template is a static member function template, then the value returned by any given specialization of this conversion function template is the address of the corresponding function call operator template specialization. Otherwise, the value returned by any given specialization of this conversion function template is the address of a function `F` that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed instance of the closure type. `F` is a constexpr function if the corresponding specialization is a constexpr function and an immediate function if the function call operator template specialization is an immediate function.

[*Note 5:* This will result in the implicit instantiation of the generic lambda's body. The instantiated generic lambda's return type and parameter types are required to match the return type and parameter types of the pointer to function. — *end note*]

[*Example 7:*

```
auto GL = [](auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL;        // OK, through conversion function template
GL_int(3);                      // OK, same as GL(3)
```

— *end example*]

13. The conversion function or conversion function template is public, constexpr, non-virtual, non-explicit, const, and has a non-throwing exception specification.

[*Example 8:*

```
auto Fwd = [](int (*fp)(int), auto a) { return fp(a); };
auto C = [](auto a) { return a; };

static_assert(Fwd(C,3) == 3);   // OK

// No specialization of the function call operator template can be constexpr (due to the local static).
auto NC = [](auto a) { static int s; return a; };
static_assert(Fwd(NC,3) == 3);  // error
```

— *end example*]

§ 4.2. Clang's approach

Modify [expr.prim.lambda.general]/6 as follows:

6. A lambda is a *generic lambda* if the *lambda-expression* has any generic parameter type placeholders ([dcl.spec.auto]), or if the lambda has a *template-parameter-list*.

[*Example 4:*

```
~~int i = [](int i, auto a) { return i; }(3, 4);                    // OK, a generic lambda~~
~~int j = []<class T>(T t, int i) { return i; }(3, 4);              // OK, a generic lambda~~
auto x = [](int i, auto a) { return i; };                           // OK, a generic lambda
auto y = [](this auto self, int i) { return i; };                   // OK, a generic lambda
auto z = []<class T>(int i) { return i; };                          // OK, a generic lambda
```

— *end example*]

Modify [expr.prim.lambda.closure]/9 as follows:

9. The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage having the same parameter and return types as the closure type's function call operator, except that if the function call operator has an explicit object parameter of type `T`, then the function type has a leading parameter of type `T`. The conversion is to "pointer to noexcept function" if the function call operator has a non-throwing exception specification. If the function call operator is a static member function or explicit object member function, then the value returned by this conversion function is the address of the function call operator. Otherwise, the value returned by this conversion function is the address of a function `F` that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type. `F` is a constexpr function if the function call operator is a constexpr function and an immediate function if the function call operator is an immediate function.

10. For a generic lambda with no *lambda-capture*, the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list, and the pointer to function has the same parameter types, as the function call operator template, except that if the function call operator template has an explicit object parameter of type `T`, then the function type has a leading parameter of type `T`. The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization.

11. [*Note 4:* If the generic lambda has no *trailing-return-type* or the *trailing-return-type* contains a placeholder type, return type deduction of the corresponding function call operator template specialization has to be done. The corresponding specialization is that instantiation of the function call operator template with the same template arguments as those deduced for the conversion function template. Consider the following:

```
auto glambda = [](auto a) { return a; };
int (*fp)(int) = glambda;
```

The behavior of the conversion function of `glambda` above is like that of the following conversion function:

```
struct Closure {
  template<class T> auto operator()(T t) const { /* ... */ }
  template<class T> static auto lambda_call_operator_invoker(T a) {
    // forwards execution to operator()(a) and therefore has
    // the same return type deduced
    /* ... */
  }
  template<class T> using fptr_t =
     decltype(lambda_call_operator_invoker(declval<T>())) (*)(T);

  template<class T> operator fptr_t<T>() const
    { return &lambda_call_operator_invoker; }
};
```

— *end note*]

[*Example 6:*

```
void f1(int (*)(int))   { }
void f2(char (*)(int))  { }

void g(int (*)(int))    { }     // #1
void g(char (*)(char))  { }     // #2

void h(int (*)(int))    { }     // #3
void h(char (*)(int))   { }     // #4

auto glambda = [](auto a) { return a; };
f1(glambda);                    // OK
f2(glambda);                    // error: ID is not convertible
g(glambda);                     // error: ambiguous
h(glambda);                     // OK, calls #3 since it is convertible from ID
int& (*fpi)(int*) = [](auto* a) -> auto& { return *a; };        // OK
```

— *end example*]

12. If the function call operator template is a static member function template or explicit object member function, then the value returned by any given specialization of this conversion function template is the address of the corresponding function call operator template specialization. Otherwise, the value returned by any given specialization of this conversion function template is the address of a function `F` that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed

instance of the closure type. `F` is a constexpr function if the corresponding specialization is a constexpr function and an immediate function if the function call operator template specialization is an immediate function.

[*Note 5:* This will result in the implicit instantiation of the generic lambda's body. The instantiated generic lambda's return type and parameter types are required to match the return type and parameter types of the pointer to function. — *end note*]

[*Example 7:*

```
auto GL = [](auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL;        // OK, through conversion function template
GL_int(3);                      // OK, same as GL(3)
```

— *end example*]

13. The conversion function or conversion function template is public, constexpr, non-virtual, non-explicit, const, and has a non-throwing exception specification.

[*Example 8:*

```
auto Fwd = [](int (*fp)(int), auto a) { return fp(a); };
auto C = [](auto a) { return a; };

static_assert(Fwd(C,3) == 3);   // OK

// No specialization of the function call operator template can be constexpr (due to the local static).
auto NC = [](auto a) { static int s; return a; };
static_assert(Fwd(NC,3) == 3);  // error
```

— *end example*]

§ 4.3. MSVC's approach

NOTE: This proposed wording doesn't explain why `[](this auto){}()` should prefer to call the user-defined `operator()` instead of using the builtin operator on the result of the lambda's non-template conversion function. I'm hoping someone at Microsoft can shed light on how MSVC tiebreaks this internally.

Modify [expr.prim.lambda.general]/6 as follows:

6. A lambda is a *generic lambda* if the *lambda-expression* has any generic non-object parameter type placeholders ([dcl.spec.auto]), or if the lambda has a *template-parameter-list*.

[*Example 4:*

```
~~int i = [](int i, auto a) { return i; }(3, 4);                    // OK, a generic lambda~~
~~int j = []<class T>(T t, int i) { return i; }(3, 4);              // OK, a generic lambda~~
auto w = [](int i, auto a) { return i; };                           // OK, a generic lambda
auto x = [](this auto self, int i) { return i; };                   // OK, a non-generic lambda
auto y = [](this auto self, auto a) { return i; };                  // OK, a generic lambda
auto z = []<class T>(int i) { return i; };                          // OK, a generic lambda
```

— *end example*]

Modify [expr.prim.lambda.closure]/4 as follows:

> The closure type for a *lambda-expression* has a public inline function call operator ~~(for a non-generic lambda)~~ or function call operator template ~~(for a generic lambda)~~ ([over.call]) whose parameters and return type are those of the *lambda-expression*'s *parameter-declaration-clause* and *trailing-return-type* respectively, and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any. The *requires-clause* of the function call operator template is the *requires-clause* immediately following **<** *template-parameter-list* **>**, if any. The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* of the *lambda-declarator*, if any.
>
> [*Note 2:* The function call operator template ~~for a generic lambda~~ can be an abbreviated function template ([dcl.fct]). — *end note*]

Modify [expr.prim.lambda.closure]/9 as follows:

9. The closure type for a non-generic *lambda-expression* with no *lambda-capture* whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage having the same parameter and return types as the closure type's function call operator (omitting the object parameter, if any). The conversion is to "pointer to noexcept function" if the function call operator has a non-throwing exception specification. If the function call operator is a static member function, then the value returned by this conversion function is the address of the function call operator. Otherwise, the value returned by this conversion function is the address of a function `F` that, when invoked, has the same effect as invoking the closure type's function call operator on a default-constructed instance of the closure type. `F` is a constexpr function if the function call operator is a constexpr function and an immediate function if the function call operator is an immediate function.

10. For a generic lambda with no *lambda-capture*, the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list as the function call operator template (omitting the invented *template-parameter* corresponding to the function call operator's explicit object parameter, if any), and the pointer to function has the same parameter types as the function call operator template (omitting the object parameter, if any). The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization.

11. [*Note 4:* If the generic lambda has no *trailing-return-type* or the *trailing-return-type* contains a placeholder type, return type deduction of the corresponding function call operator template specialization has to be done. The corresponding specialization is that instantiation of the function call operator template with the same template arguments as those deduced for the conversion function template. Consider the following:

```
auto glambda = [](auto a) { return a; };
int (*fp)(int) = glambda;
```

The behavior of the conversion function of `glambda` above is like that of the following conversion function:

```
struct Closure {
  template<class T> auto operator()(T t) const { /* ... */ }
  template<class T> static auto lambda_call_operator_invoker(T a) {
    // forwards execution to operator()(a) and therefore has
    // the same return type deduced
    /* ... */
  }
  template<class T> using fptr_t =
     decltype(lambda_call_operator_invoker(declval<T>())) (*)(T);

  template<class T> operator fptr_t<T>() const
    { return &lambda_call_operator_invoker; }
};
```

— *end note*]

[*Example 6:*

```
void f1(int (*)(int))   { }
void f2(char (*)(int))  { }

void g(int (*)(int))    { }     // #1
void g(char (*)(char))  { }     // #2

void h(int (*)(int))    { }     // #3
void h(char (*)(int))   { }     // #4

auto glambda = [](auto a) { return a; };
f1(glambda);                    // OK
f2(glambda);                    // error: ID is not convertible
g(glambda);                     // error: ambiguous
h(glambda);                     // OK, calls #3 since it is convertible from ID
int& (*fpi)(int*) = [](auto* a) -> auto& { return *a; };        // OK
```

— *end example*]

12. If the function call operator template is a static member function template, then the value returned by any given specialization of this conversion function template is the address of the corresponding function call operator template specialization. Otherwise, the value returned by any given specialization of this conversion function template is the address of a function `F` that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed instance of the closure type. `F` is a

constexpr function if the corresponding specialization is a constexpr function and an immediate function if the function call operator template specialization is an immediate function.

[*Note 5:* This will result in the implicit instantiation of the generic lambda's body. The instantiated generic lambda's return type and non-object parameter types are required to match the return type and parameter types of the pointer to function. — *end note*]

[*Example 7:*

```
auto GL = [](auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL;        // OK, through conversion function template
GL_int(3);                      // OK, same as GL(3)
```

— *end example*]

13. The conversion function or conversion function template is public, constexpr, non-virtual, non-explicit, const, and has a non-throwing exception specification.

[*Example 8:*

```
auto Fwd = [](int (*fp)(int), auto a) { return fp(a); };
auto C = [](auto a) { return a; };

static_assert(Fwd(C,3) == 3);   // OK

// No specialization of the function call operator template can be constexpr (due to the local static).
auto NC = [](auto a) { static int s; return a; };
static_assert(Fwd(NC,3) == 3);  // error
```

— *end example*]

§ 5. Straw polls taken in EWG, 2023-11-07

	SF	***F***	***N***	***A***	***SA***
Move forward with the option to "punt" from D3031, forwarding to CWG for C++26. A subsequent paper can explore other solutions.	*9*	*11*	*3*	*0*	*0*